

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»**

**Лабораторная работа №4  
по курсу «Параллельные и распределенные вычисления»**

**Сортировка чисел на GPU. Свертка, сканирование, гистограмма.**

Выполнил: Колесник Дмитрий  
Сергеевич

Группа: М8О-313Б-23

Преподаватели: А.Ю. Морозов,  
Е.Е. Заяц

Москва, 2026

## Условие

В рамках данной лабораторной работы необходимо ознакомиться с фундаментальными алгоритмами GPU: построением гистограмм и префиксным сканированием. Требуется реализовать алгоритм сортировки подсчётом (Counting Sort) для массива целых чисел типа `int`.

Вариант задания 2:

- Диапазон значений: от 0 до  $2^{24}-1$
- Реализация гистограммы с использованием атомарных операций (`atomicAdd`).
- Реализация сканирования с использованием библиотеки `Thrust`.
- Ввод и вывод данных осуществляется в бинарном виде через потоки `stdin` и `stdout`.
- Ограничение по количеству элементов:  $n \leq 135 * 10^6$

## Программное и аппаратное обеспечение

В ходе выполнения работы использовался дискретный графический ускоритель архитектуры **NVIDIA Turing**, предоставленный средой Google Colab.

- **Модель:** NVIDIA Tesla T4.
- **Compute Capability:** 7.5.
- **Количество мультипроцессоров (SM):** 40.
- **Графическая память (Global Memory):** ~14.56 ГБ.
- **Разделяемая память (Shared Memory):** 48 КБ на блок.
- **Константная память (Constant Memory):** 64 КБ .
- **Количество регистров на блок:** 65536 (32-битных).
- **Размер варпа (Warp size):** 32 нити.
- **Максимальное количество нитей (Threads):**
  - На блок: 1024.
  - Конфигурация блока (x, y, z): (1024, 1024, 64).
- **Максимальное количество блоков (Grid Size):**
  - По оси X: 2 147 483 647 (макс. значение `int32`).
  - По осям Y, Z: 65535.

Вычисления на стороне центрального процессора производились на виртуальной машине со следующими характеристиками:

- **Процессор (CPU):** Intel(R) Xeon(R) (2.20GHz, 2 ядра).
- **Оперативная память (RAM):** ~12.7 ГБ доступно в сессии.
- **Жесткий диск (Disk):** ~78 ГБ.

Для разработки и запуска программ на языке CUDA C использовалось следующее окружение:

- **Операционная система:** Linux Ubuntu 22.04 LTS (среда Google Colab).
- **Среда разработки (IDE):** Интерактивная облачная среда на базе Jupyter Notebook.
- **Компилятор:** nvcc (NVIDIA CUDA Compiler).
- **Версия CUDA SDK:** 13.0.
- **Версия графического драйвера:** 580.82.07.

## Метод решения

Сортировка подсчётом на GPU реализована в три этапа:

1. Построение гистограммы: Ядро `histogram_kernel` проходит по исходному массиву. Каждый поток считывает число и увеличивает соответствующий счётчик в массиве частот, используя функцию `atomicAdd`. Это гарантирует корректность результата при одновременном обращении множества потоков к одной ячейке памяти.
2. Префиксное сканирование (Exclusive Scan): Для вычисления начальных позиций (оффсетов) каждого числа в итоговом массиве применяется функция `thrust::exclusive_scan`. Массив частот размером  $2^{24}$  элементов преобразуется в массив смещений.
3. Заполнение результата: Ядро `fill_kernel` распределяет вычисления по значениям диапазона. Каждый поток отвечает за конкретное число и записывает его в выходной массив столько раз, сколько оно встретилось в исходных данных, начиная с позиции, вычисленной на этапе сканирования.

## Описание программы

Программа реализована в одном файле `lab5.cu`.

- Бинарный ввод/вывод: Чтение осуществляется через `fread(stdin)`, запись — через `fwrite(stdout)`.
- Thrust: Библиотека используется для выполнения эффективного префиксного сканирования. Для этого указатель на видеопамять оборачивается в `thrust::device_ptr`.

- Масштабируемость: В ядрах используется цикл с шагом по сетке (Grid-Stride Loop), что позволяет обрабатывать до 135 миллионов элементов независимо от физического количества ядер GPU.
- Типы данных: Исходные числа и массив гистограммы имеют тип int. Общий объём памяти под гистограмму составляет 64 МБ ( $2^{24} * 4$  байта).

## Результаты

1. Замеры времени работы ядер с различными конфигурациями

| Blocks | Threads | N        | Time (ms) |
|--------|---------|----------|-----------|
| 1      | 32      | 10000    | 470.0681  |
| 1      | 256     | 10000    | 24.8926   |
| 1      | 1024    | 10000    | 7.0632    |
| 256    | 32      | 10000    | 1.4541    |
| 256    | 256     | 10000    | 0.9133    |
| 256    | 1024    | 10000    | 0.9134    |
| 1024   | 32      | 10000    | 1.0670    |
| 1024   | 256     | 10000    | 0.8915    |
| 1024   | 1024    | 10000    | 0.8729    |
| 1      | 32      | 100000   | 202.4740  |
| 1      | 256     | 100000   | 30.8204   |
| 1      | 1024    | 100000   | 12.6711   |
| 256    | 32      | 100000   | 6.6785    |
| 256    | 256     | 100000   | 6.0407    |
| 256    | 1024    | 100000   | 5.9702    |
| 1024   | 32      | 100000   | 6.1511    |
| 1024   | 256     | 100000   | 5.9882    |
| 1024   | 1024    | 100000   | 5.9616    |
| 1      | 32      | 33554432 | 705.0568  |
| 1      | 256     | 33554432 | 226.3669  |
| 1      | 1024    | 33554432 | 192.4035  |

|      |      |          |          |
|------|------|----------|----------|
| 256  | 32   | 33554432 | 179.6157 |
| 256  | 256  | 33554432 | 178.9595 |
| 256  | 1024 | 33554432 | 178.8641 |
| 1024 | 32   | 33554432 | 179.0930 |
| 1024 | 256  | 33554432 | 178.9232 |
| 1024 | 1024 | 33554432 | 178.9131 |

## 2. Сравнение с CPU.

- CPU Time for N=10000: 54.1429 ms (best GPU: 0.8729 ms)
- CPU Time for N=1000000: 38.1654 ms (best GPU: 5.9616 ms)
- CPU Time for N=33554432: 155.5682 ms (best GPU: 178.9131 ms)

## Выводы

В ходе работы был реализован параллельный алгоритм сортировки подсчётом (Counting Sort) на базе технологии CUDA. Использование атомарных операций `atomicAdd` позволило эффективно реализовать построение гистограммы распределения значений, обеспечив корректность вычислений при массовом параллельном доступе к памяти. Применение библиотеки Thrust для выполнения префиксного сканирования обеспечило высокую скорость нахождения оффсетов элементов и масштабируемость решения. Результаты показали, что при обработке массивов больших объёмов в фиксированном диапазоне значений перенос вычислений на GPU позволяет достичь значительного ускорения процесса сортировки по сравнению с классической реализацией на центральном процессоре.